

# MONOPOLY.LK

## VIVA PREPARATION GUIDE

---

C language console simulation of Monopoly,  
adapted to Sri Lankan locations and rules.

This guide explains the whole project in plain language:  
what it does, how it is built, and model answers to the  
questions an examiner is most likely to ask.

Read order: Section 1 (memorise) -> Section 2 (whole project)  
-> Section 3 (file by file) -> Section 4 (Q&A practice)  
-> Section 5 (numbers to remember) -> Section 6 (glossary)

No source file was changed to make this guide.

# 1. The Ten Answers to Memorise

---

If a viva catches you cold, these simple answers cover most questions. Read each one aloud once, then try to say it back without looking.

## Q: What is this project?

A: It is a complete simulation of the board game Monopoly, written in C, adapted to Sri Lanka - cities like Colombo, Kandy, Galle and Jaffna instead of London. Four computer players with different personalities (aggressive, conservative, risk-taker, trader) play against each other.

Every rule is implemented: buying and renting property, building houses and hotels, mortgages, bank loans, taxes, jail, auctions, insurance, disasters, inflation, market booms and declines.

The whole game prints to the console - there is no graphics.

*MEMORISE / FLAG: One line: 'A console simulation of Sri Lankan Monopoly where 4 AI players with different strategies play 500 rounds of game rules.'*

## Q: How is the code organised?

A: One .c file per responsibility: main.c drives the program, board.c builds the 40-square board, players.c holds the 4 strategies, finance.c handles money and property, game.c is the game loop, and events.c / economy.c / market.c / bank.c / auction.c / insurance.c handle the rest.

All shared data lives in types.h and every function is declared in functions.h.

*MEMORISE / FLAG: Pretend you are explaining a toolbox: 'each file is one tool for one job'.*

## Q: Why do all functions take a 'GameState game[]' parameter?

A: There is exactly ONE GameState created, in main. Passing it as an array to every function is the C idiom meaning 'pass by reference' - the function reads and changes the real game, not a copy.

It avoids using global variables (clean, no hidden state) and avoids pointer syntax.

*MEMORISE / FLAG: Keywords to say: single GameState, no globals, array-passed so changes persist.*

## Q: What is the main loop of the game?

A: Players roll two dice and move around the board.

If they land on a property they may buy it, otherwise it is auctioned; if it is owned they pay rent.

Taxes, events, the bank and insurance squares do their own action.

Then the player may build houses/hotels and handle mortgages.

A 'round' finishes when every surviving player has passed GO at least once.

Game ends when one player is left or at round 500; winner = highest net worth.

*MEMORISE / FLAG: Say: 'roll, move, land, pay/collect, build, then round summary'.*

## Q: How is the winner decided?

A: The last solvent player wins. If several survive to round 500, the winner is the player with the highest net worth, where net worth = cash + property value + building value - outstanding loan.

*MEMORISE / FLAG: Memorise the net worth formula exactly.*

## Q: How is randomness used?

A: rand() generates all randomness: dice (rand()%6+1 for each die), events, disasters, inflation. srand((unsigned)time(NULL)) seeds it once at the start so every game is different.

## Q: What data structures are used?

A: Structs (Property, Player, Square, Loan, Economy, GameState, ActiveModifier), arrays (board[40], players[4], modifiers[48]) and enums (SquareType, PropertyGroup, PlayerType, InsuranceType, DisasterType, ModifierType, ModifierSource). No dynamic memory.

*MEMORISE / FLAG: If asked 'any malloc?': 'No - everything is fixed size, decided at compile time.'*

## Q: What are the four strategies?

A: AGGRESSIVE\_INVESTOR - buys to complete monopolies even under its reserve, always borrows, builds eagerly.

CONSERVATIVE\_BANKER - cautious: avoids property in a recession, never builds hotels with a loan, keeps a cash cushion.

RISK\_TAKER - spends everything, never pays bail, sells cheap property to survive, only insures after a loss.

OPPORTUNISTIC\_TRADER - buys cheap, sells anything expected to lose value, trades market conditions.

*MEMORISE / FLAG: Remember one word each: 'monopoly-hunter', 'safe-cautious', 'all-in', 'buy-low-sell-early'.*

**Q: What does a player have to track (the Player struct)?**

A: Name, strategy, board position, cash, jail state, bankrupt flag, counts of properties/railways/utilities owned, one Loan, and two small strategy flags (sufferedLoss, antiSpecRounds).

**Q: Give one rule implemented that makes this different from normal Monopoly.**

A: Answer options (pick whichever you remember best):

- Market reviews every 10 rounds: one colour group booms, another declines (timed effects).
- Bank loans: you may borrow up to 75% of collateral, interest accrues every round, overdue loans are foreclosed.
- Anti-Speculation Act: no more than 3 undeveloped properties for 5 rounds, or the excess is force-auctioned.
- Buildings age: condition drops, neglect causes structural damage, properties depreciate past age 50.
- Insurance and disasters: every 10 rounds a developed property is hit by fire/flood/riot/etc.

## 2. The Whole Project in Plain English

---

### 2.1 What actually happens when you run it

- `main()` creates the single `GameState` (all zeroed) and calls `startGame()`.
- `startGame()` seeds the random generator, builds the board, creates the 4 players, initialises the economy and market, then prints each player's starting money (LKR 30,000).
- `determineTurnOrder()` rolls dice for everyone; ties are re-rolled until there is a strict order.
- `playGame()` runs the loop: players take turns until everyone has passed GO (that is one round).
- Between rounds: loans accrue interest, insurance policies tick, properties age, event timers tick, and every 10 / 15 / 20 rounds the economy does disasters, inflation, market reviews, economic events, regional cards and government regulations.
- The game stops when only one solvent player remains (or after 500 rounds) and `displayFinalResults()` prints the winner and final standings.

### 2.2 The board (40 squares)

- 22 colour-group properties across 8 groups: Brown (2), Light Blue (3), Pink (3), Orange (3), Red (3), Yellow (3), Green (3), Dark Blue (2).
- 4 railways (Colombo Fort, Kandy, Galle, Jaffna) and 2 utilities (CEB electricity, water board).
- 12 special squares: GO, Community Development Fund, Income Tax, three National Event squares, Jail / Just Visiting, two Insurance squares, Free Parking, Go To Jail, and Bank of Ceylon.
- Every square stores its name, type and (for property) all the property fields in a `Property` struct.
- The same `Property` struct is reused for railways/utilities/special squares - only the fields that matter are used.

### 2.3 What a single turn does (`playTurn`)

- 1. Auto-repair any damaged buildings, perform maintenance, fix structural damage (if affordable).
- 2. If the player is in jail, `handleJail()` runs (doubles release, or pay bail, or stay; max 3 turns).
- 3. Roll two dice, move forward by that many squares (wrapping 0-39); passing GO collects LKR 2,000.
- 4. Resolve the landing square: rent + possible purchase, or tax, or event card, or bank visit, or insurance visit, or Go To Jail.
- 5. Build buildings if affordable and wanted (`constructBuildings`).
- 6. The Opportunistic Trader may sell declining properties.
- 7. Mortgage / redeem mortgages if cash is low.

### 2.4 Money and value rules

- Net worth = cash + property value + building value - outstanding loan.
- Property value uses `currentMarketValue()`: purchase price minus age depreciation, then multiplied by all active value modifiers.
- Building value = sum of each property's houses x house cost, hotels x hotel cost.
- Income Tax = 15% of net worth; Community Development Fund = 10% of property assets only (the rate itself drifts with the economy).
- Mortgaging returns the property's mortgage value (roughly half price); redeeming costs 110% of it.
- Bankruptcy: when cash goes negative the player is declared bankrupt; buildings are demolished and every owned square is auctioned; the debt is written off.

### 2.5 The modifier system - the most important idea

- Events, economic events, regulations, market booms/declines and regional cards all ADD a timed entry into `Economy.modifiers[]` (up to 48 entries).
- Each entry: type, scope (colour group or square index), percent, rounds left, source.
- `modifierMultiplier()` multiplies every matching entry together (e.g. two +25% rent effects give roughly +56%). Rule-LK 34 says effects accumulate.
- `decrementModifiers()` ticks each entry down one per round and removes expired ones.
- The base purchase price, base rent and base costs are NEVER modified permanently - only the modifiers change what is currently paid/earned.

### 3. File-by-File: What Lives Where

---

For each file: one-line job description, then its functions with a short explanation. If an examiner points at any function, these sentences are your answer.

#### **main.c - the entry point - only 17 lines**

- Includes stdio.h (printf), types.h and functions.h.
- Creates GameState game[1] = {0} - 'the only place a GameState is ever created'.
- Prints the MONOPOLY-LK SIMULATION banner and calls startGame(game).

#### **types.h - all constants and data types**

- #define constants: BOARD\_SIZE 40, MAX\_PLAYERS 4, START\_MONEY 30000, MAX\_ROUNDS 500, GO\_MONEY 2000, JAIL\_BAIL 300, plus rent tables, loan rules, insurance rates, auction rules, depreciation and anti-speculation numbers.
- Enums: SquareType, PropertyGroup, PlayerType, InsuranceType, DisasterType, ModifierType, ModifierSource.
- Structs: Loan, Property, Square, Player, ActiveModifier, Economy, GameState.
- GameState bundles everything: Square board[40], Player players[4], Economy economy.

#### **functions.h - declares every function, grouped by file**

- Header guards (#ifndef FUNCTIONS\_H ... #endif) prevent double inclusion.
- Each section is marked with /\* board.c \*/, /\* players.c \*/ and so on.
- Any module can call any other module because all prototypes are visible here.

#### **board.c - builds the 40-square Sri Lankan board**

- setProperty(): fills all fields of a colour property (price, rent, mortgage, house/hotel cost, group, owner=-1).
- setRailway() / setUtility(): price saved, mortgage = price/2, extra building fields zeroed.
- setSpecialSquare(): GO, tax, jail, insurance, bank, event squares; owner forced to -1 so it is not 'owned'.
- initializeBoard(): calls the setters in order - special squares, railways, utilities, then all 22 properties by colour group.

#### **players.c - the four personality strategies (decision functions return 1/0)**

- initializePlayers(): names, strategies, position 0, cash 30,000, zero counters, resets loans.
- shouldBuyProperty(): per-strategy rule using cash and 1000 reserve / half cash / price limits.
- shouldConstruct(): decides a house/hotel purchase; conservative banker never hotels with a loan.
- wantsLoan / wantsToRepayLoan / wantsIncreaseLoan / wantsExtendLoan / wantsRefinance: loan decisions per strategy.
- desiredInsurance(): which policy each strategy buys (or none).
- willingToBid(): auction ceiling per strategy (120% / below value / anything / 80%).
- shouldMortgage / shouldRedeemMortgage / shouldPayBail: cash-pressure decisions.
- shouldMaintain() / shouldRenovateAgeDepreciation(): maintenance spending rules.

#### **finance.c - money, rent, property purchases, mortgages, bankruptcy**

- stripOwnership(): owner=-1, un-mortgage, unlock collateral, remove insurance.
- resetLoan() / adjustOwnedCount(): clear a loan; update property/railway/utility counts.
- receiveMoney() / payMoney(): the only places cash changes; payMoney() can trigger bankruptcy.
- payTax() / payCommunityFundTax(): 15% of net worth / 10% of property assets (rate scaled).
- buyProperty(): if the player wants it and can afford it, buy; otherwise the property is auctioned.
- payRent(): owner gets rent (or no rent if mortgaged / damaged / business-interrupted / closed).
- calculateRent(): base rent x (1,2,3,5,7) for 0-4 houses, x10 for a hotel, then modifiers and condition.
- calculateRailwayRent() / calculateUtilityRent(): tables 250/500/1000/2000; 4x/10x dice.
- mortgage / redeem / handleMortgageDecisions: raise cash or clear mortgages.
- calculateNetWorth / calculatePropertyValue / calculateBuildingValue: the valuation trio.
- developGroup() / constructBuildings(): even building; hotels need the whole group at 4 houses.
- enforceAntiSpeculation() / forceSellExcess() / sellUndevelopedPropertyToAuction(): the Anti-Speculation Act.
- sellLowValueProperty() / sellDecliningProperties(): Risk Taker rescue sale / trader's early exit.
- wouldCompleteMonopoly(): checks whether buying completes a colour group.

## game.c - the game loop, dice, turns, jail, turn order

- rollDice(): rand()%6+1 twice, adds them.
- movePlayer(): (old+dice)%40; if the wrap happens, pay GO money LKR 2,000.
- handleJail(): doubles release (no move that turn), bail if the strategy wants, forced bail after 3 turns.
- playTurn(): the 7 steps of one player's turn (see 2.3).
- resolveGroup(): rolls everyone, sorts descending, re-rolls tied runs (recursion).
- determineTurnOrder(): builds the final strict order.
  - playGame(): the round loop - keeps playing until everyone has passed GO, then runs all between-round bookkeeping.
- displayRoundSummary(): prints cash, net worth, properties, loans per player each round.
- countSolventPlayers() / countHouses() / countHotels(): small helpers.
- determineWinner(): highest net worth among surviving players (with an everyone-bankrupt fallback).
- displayFinalResults(): winner + full final standings.

## events.c - national event cards, economic events, government regulations

- executeEvent(): draws one of 20 National Event Cards (Tourism Hype, Fuel Shortage, Heavy Floods, Political Rally, Stock Market Rise/Downturn, Housing Subsidy, interest changes, Tax Amnesty, Power Failure, Port Expansion, Festival Season, Labour Strike, Insurance Discount, Property Revaluation, Currency Depreciation, Government Grant, National Disaster).
- The card index cycles 0-19 and wraps - like 'draw the top card and put it at the bottom'.
- triggerEconomicEvent(): 8 economy-wide events every 15 rounds (boom, crisis, monsoon, recession, stock boom, housing programme, foreign investment, political unrest).
  - triggerGovernmentRegulation(): 8 regulations every 20 rounds (tax changes, subsidies, tariff revisions, luxury property tax, Anti-Speculation Act...).
- decrementEventTimers(): ticks the non-modifier timers (closed property, construction suspension).

## economy.c - the economy engine: inflation, modifiers, value, ageing, maintenance

- initEconomy(): starts loan interest at 8%, card index 0, no modifiers.
- applyRate(old, rate):  $\text{old} + \text{old} \times \text{rate} / 100$ , never below 1 - the inflation formula.
- addModifier() / addSourcedModifier(): put timed effects into the modifier array.
- modifierMultiplier(): multiplies all matching modifiers; 100 = no effect.
- isModifierActive(): for flag-style effects (flood risk, recession).
- currentTaxRatePercent(): base rate scaled by global value modifiers.
- decrementModifiers(): age and remove expired modifiers.
- formatLKR(): prints money with thousands separators (30000 -> 30,000).
- applyInflation(): picks -3,0,2,5,8,12%, applies to prices, rents, costs and loan interest.
- currentMarketValue(): purchase price - depreciation, then x all value modifiers.
- ageProperties(): properties age 1 per round; past 50 they depreciate 5% every 5 rounds (max 30%).
- tryRenovateAgeDepreciation(): pay 10% of value to reset age and boost rent by 5%.
- ageBuildings(): developed buildings lose 2 condition per round; 20 neglected rounds cause structural damage.
- performMaintenance() / renovateStructuralDamage(): restore condition; fix structural damage at 150%.

## market.c - the property market: boom/decline cycles and regional cards

- initMarket(): clears group cooldowns and last boom/decline groups.
- groupName(): maps a colour-group enum to its name.
- pickEligibleGroup(): picks a random group not on cooldown and not just used.
- reviewPropertyMarket(): every 10 rounds one group booms (+20% value, +25% rent, +15% purchase/mortgage) and another declines (-15% value, -20% rent, -10% mortgage, -25% auction start); cooldown 30 rounds.
  - drawRegionalCard(): 12 regional development cards (Southern Tourism Boom, Port City Expansion, IT Growth, Northern Development, Tea Export Boom, Airport Expansion, University Growth, Beach Pollution, Flood Damage, Transport Strike, Tariff Increase, Water Shortage).
- displayMarketConditions(): shows Boom / Decline / Regional / Other conditions, inflation, loan interest and income tax at the end of each round.

## bank.c - loans and collateral

- `groupBasePrice()`: loan valuations use the group's first property price.
- `totalEligibleCollateral()`: sum of un-mortgaged, unlocked property values (market-adjusted).
- `calculateMaxLoan()`: 75% of eligible collateral.
- `obtainLoan()`: one loan at a time; duration 20 rounds; marks all collateral `loanLocked`.
- `repayLoan()`: partial or full; full repayment releases collateral.
- `handleBankVisit()`: decide between repay / increase / extend / refinance / obtain.
- `increaseLoan()` / `extendLoan()` / `refinanceLoan()`: borrow more, add 10 rounds, switch to current rate.
- `foreclose()`: overdue loan - pledged properties are demolished, auctioned; player may go bankrupt.
- `processLoans()`: each round adds interest to the principal and counts down; expires -> foreclose.

### **auction.c - selling property by auction**

- `getAskingValue()`: current market value adjusted by any 'auction price' modifier.
- `runAuction()`: opening bid = asking/2, every bid +250, a declining bidder is out for good, max 200 safety rounds; a forced seller (Anti-Speculation) never bids.
- Winner pays and takes ownership; if there is a seller (not the bank), the seller receives the money.

### **insurance.c - disasters, policies, claims and repairs**

- `propertyValue()` = `currentMarketValue()`; `repairCost()` = 30% of it.
- `isCovered()`: Basic covers Fire/Flood; Comprehensive and Business Interruption cover all disasters.
- `compensationPercent()`: Basic 80%, Comprehensive/Business Interruption 100%.
- `pickDisaster()`: weighted choice - monsoon/riots boost flood/riot chances.
- `purchaseInsurance()`: premium = 5%/10%/15% of value (scaled by insurance modifiers), 20 rounds.
- `handleInsuranceVisit()`: renew a near-expiring policy or buy for an uninsured suitable property.
- `triggerDisaster()`: every 10 rounds a random developed property is hit; claims pay compensation; uninsured owners suffer the full cost and stop earning rent until repaired.
- `payCompensation()` / `disasterName()`: claim payout (Business Interruption loses 5 income rounds) and names.
- `tryAutoRepair()`: at the start of each turn, owners repair damage they can afford.
- `processInsuranceExpiry()`: ticks policies down; warns at 3 rounds left, then expires.

## 4. Likely Viva Questions and Model Answers

### 4.1 About the C language

#### Q: What is the difference between a struct and an enum?

A: A struct groups several related variables (fields) into one type - e.g. Property groups name, group, price, rent, houses, insurance...

An enum defines a set of named integer constants - e.g. PropertyGroup = {BROWN, LIGHT\_BLUE, ...}. Enums make the code readable instead of using magic numbers like 0,1,2.

*MEMORISE / FLAG: struct = custom container type; enum = named list of constants.*

#### Q: What is a function prototype (declaration)?

A: It tells the compiler a function's name, return type and parameter list before it is used. functions.h contains every prototype, so main.c can call startGame() even though it is defined in game.c.

*POSSIBLE FOLLOW-UP: Why would you get an error without prototypes? -> 'implicit declaration' warning/error.*

#### Q: What do the header guards #ifndef / #define / #endif do?

A: They make sure a header is only compiled once. If types.h were included twice, the structs would be defined twice, which is a compile error. The guard prevents that.

#### Q: What is a pointer? Does this project use any?

A: A pointer stores the memory address of another variable. This project deliberately avoids pointer syntax: GameState is passed as an array parameter (game[]) which is the C way of passing by reference, and all access is game[0].players[0].cash etc.

Note: arrays 'decay' to pointers when passed, so game[] is effectively a pointer to the first GameState.

*MEMORISE / FLAG: Safe answer: 'We engineered it to avoid pointers; the array syntax is our reference style.'*

#### Q: What does (void)playerIndex; mean in executeEvent()?

A: It casts an unused parameter to void, which tells the compiler 'yes, I know this parameter is unused' and suppresses the unused-parameter warning.

#### Q: How does rand() work and why seed it?

A: rand() returns a pseudo-random number from a fixed sequence. The sequence is the same every run unless the seed changes. srand((unsigned)time(NULL)) seeds it with the current time so each game is different.

*MEMORISE / FLAG: If asked 'why % 6 + 1': dice need 1..6 - rand()%6 gives 0..5, so add 1.*

#### Q: What do %d, %s, %+d and %% print?

A: %d prints an int, %s a string, %+d forces a plus sign for positive numbers (used for inflation like +8%%), and %% prints one literal percent character.

#### Q: What is the difference between = and ==?

A: = assigns a value; == compares values. Using the wrong one is a classic bug. In this code == is used in conditions such as if(owner == -1).

#### Q: Why is there no dynamic memory (malloc/free)?

A: Everything is fixed-size by design: 40 squares, 4 players, 48 modifiers. Stack arrays need no manual memory management, so there are no leaks and no NULL checks.

#### Q: What is recursion? Give an example from the code.

A: A function that calls itself. resolveGroup() in game.c re-calls itself on the small group of players who rolled a tie, until the order is strict. This is clean recursion with a base case (run of length 1).

#### Q: How does the code avoid infinite loops?

A: Several safety caps: constructBuildings() has a 'safetyLimit < 20', the auction has AUCTION\_MAX\_ROUNDS 200, and the forced-sale loop stops if no property could be sold (otherwise loan-locked properties would trap it forever).

*MEMORISE / FLAG: Good to volunteer: 'we wrote safety limits to guarantee termination'.*

#### Q: Why do the loops often use for(i=0; i<BOARD\_SIZE; i++)?

A: Because the board is an array of 40 squares; we scan all squares when buying, renting, auctioning, selling, ageing or checking ownership.

### 4.2 About the design

**Q: Why is there only one GameState, created in main?**

A: It centralises ALL game state (board, players, economy) in one object created exactly once. Every function takes it as a parameter, which keeps the state explicit, avoids global variables and makes each function easy to reason about.

*POSSIBLE FOLLOW-UP: What if you stored the board as a global? -> hidden dependencies, harder to test/reuse.*

**Q: Why pass game[] and access game[0]?**

A: In C, an array name in a call is a pointer to its first element, so the function modifies the real data. game[0] is that single element. It is the chosen 'reference' idiom for this project.

**Q: Explain the ActiveModifier design in the Economy struct.**

A: Instead of permanently changing prices, every timed effect is stored as a small record (type, group, index, percent, roundsLeft, source) in an array. A multiplier function combines all matching records. This is clean because effects stack and then expire.

*MEMORISE / FLAG: The rule number to quote: Rule-LK 34 - effects are cumulative and multiplied.*

**Q: Why is Money stored as an int and formatted with a custom function?**

A: Money is an integer number of LKR. formatLKR() inserts thousands separators (30000 -> 30,000) for nicer console output without floating point.

**Q: What does the Square struct look like and why the Property field inside it?**

A: Square has index, type (SquareType), name and a full Property struct. Reusing Property inside Square lets every square - even railways and special squares - carry the fields needed, so setSpecialSquare can force owner = -1.

**Q: How are the four strategies implemented without inheritance?**

A: C has no classes, so each strategy is implemented with a switch on the player's strategy enum inside each decision function (shouldBuyProperty, shouldConstruct, ...). Adding a strategy means adding a case in every decision function.

*MEMORISE / FLAG: Honest answer bonus: 'A cleaner version could use function pointers per strategy.'*

### 4.3 About specific behaviour

**Q: How does a round work? Why does a round take several turns per player?**

A: A round is NOT one turn per player. It ends only when every non-bankrupt player has passed GO at least once. Because players move different distances each turn, the game cycles through players several times until everyone has crossed GO, then does end-of-round bookkeeping.

Bookkeeping: processLoans, insurance expiry, ageing properties, ageing buildings, decrement modifiers and timers, and the periodic events.

**Q: How does turn order get decided fairly?**

A: Everyone rolls two dice. Results are sorted descending. Any group that ties rolls again (resolveGroup re-calls itself on just the tied players) until there are no ties. Then the game prints the final order.

**Q: Explain the jail rules.**

A: If a player is in jail: rolling doubles releases them (but they do not move that turn).

If doubles fail, the strategy may choose to pay bail (LKR 300) and be released (still no move).

After 3 turns in jail, the player MUST pay bail and leaves.

*MEMORISE / FLAG: Number to say: JAIL\_MAX\_TURNS = 3, JAIL\_BAIL = 300.*

**Q: What happens when a player goes bankrupt?**

A: cash dropping below zero triggers bankruptcy in payMoney().

Before that, a Risk Taker may sell its cheapest undeveloped property to the Bank to save itself.

If still negative: BANKRUPT is printed, the loan is cleared, all buildings are demolished, every owned square is stripped and auctioned, and the remaining debt is written off (cash set to 0).

**Q: How is rent calculated for a normal property?**

A: Take baseRent, multiply by an index that depends on development: no house x1, 1 house x2, 2 houses x3, 3 houses x5, 4 houses x7, hotel x10.

Then multiply by the group rent modifier, the global rent modifier, a hotel modifier if it is a hotel, and finally the building condition percentage (100/90/75/50/0) since houses/hotels are present.

**Q: How is railway and utility rent calculated?**

A: Railways: one owned = 250, two = 500, three = 1000, four = 2000 (times current modifiers).

Utilities: one owned = 4 x dice roll; both owned = 10 x dice roll.

**Q: Why does developGroup() build evenly and block hotels?**

A: Rule 9: houses must be spread evenly within a colour group (always build on the property with the fewest houses). A hotel needs every property in the group at four houses and no blocked member (mortgaged/damaged/loan-locked). Rule: never pile a 5th house on one property.

**Q: What is the Anti-Speculation Act?**

A: A government regulation (or rule-LK 8): once active, a player may hold at most 3 undeveloped properties. If they stay above the limit for 5 consecutive rounds, the excess undeveloped properties are force-sold at auction (lowest value first). Loan-locked collateral is never sold.

**Q: How do disasters and insurance interact?**

A: Every 10 rounds one random developed property is hit by a disaster chosen with weights. Repair cost = 30% of the property's value.

If insured: compensation paid now (Basic 80% for fire/flood; Comprehensive 100% for anything; Business Interruption 100% plus 5 rounds of lost rent).

If not insured: property becomes damaged, stops earning rent, and the owner repairs it as soon as they can afford it (tryAutoRepair at the start of every turn).

An uninsured loss sets the player's sufferedLoss flag - which is why a Risk Taker starts insuring only after its first disaster.

**Q: How do loans work?**

A: A player may obtain ONE secured loan at the Bank square: amount = 75% of eligible collateral; interest = the economy's current rate (starts 8%); duration = 20 rounds.

Collateral properties become loanLocked and cannot be sold, mortgaged or given to auctions.

Every round the loan accrues interest (amount grows) and the countdown drops.

On a Bank visit the player can repay, extend (+10 rounds), refinance at the current rate, or draw on newly acquired collateral.

If the term expires: foreclose - all pledged properties are demolished and auctioned; if nothing remains and cash  $\leq$  0 the player is bankrupt.

*MEMORISE / FLAG: Numbers: 75% of collateral, base 8% interest, 20 rounds, +10 extension.*

**Q: What does every 10 / 15 / 20 rounds do?**

A: 10 (ECONOMY\_CYCLE): disasters, inflation, property market review.

15 (EVENT\_CYCLE): one economic event and one regional development card.

20 (REGULATION\_CYCLE): one government regulation.

Events and modifiers are rolled together with each round's bookkeeping.

**4.4 'Read this function' questions****Q: Explain rollDice().**

A: It rolls two virtual dice: each is  $\text{rand}() \% 6 + 1$  (1 to 6), and it returns the two added together - so the sum is between 2 and 12.

*MEMORISE / FLAG: If asked 'could it be 7 with doubles?': yes - sum 2..12; doubles are detected separately in handleJail.*

**Q: Explain currentMarketValue().**

A: Start from the stored purchasePrice. Subtract age depreciation (a percentage). Then multiply by every active value modifier: global value, plus the property's GROUP value (for colour properties) or the railway value (for stations), plus any single-square index modifier.

This one function is used everywhere a real price is needed: buying, insurance, repairs, renovation, net worth - so all of those stay consistent.

**Q: Explain modifierMultiplier().**

A: It starts at 100 (meaning 'no change').

It loops through every active modifier of the requested type; only those that match the requested group/index (a -1 means 'global, matches everything') are applied by multiplying:  $\text{mult} = \text{mult} * \text{percent} / 100$ .

The result is cumulative, e.g. two +25% effects become about 156%.

**Q: Explain the Auction function flow.**

A: Opening bid = asking value / 2. All non-bankrupt players (except a forced seller) form the pool.

Each round, each active player decides whether to bid +250 using its strategy's willingToBid(); if it declines it is removed permanently.

When one bidder remains, it may take the property at the next increment or at opening price.

Winner pays; the seller (if any) receives the money; owner and owned-count are updated.

Safety limit 200 rounds guarantees it always ends.

**Q: Explain performMaintenance() and ageBuildings().**

A: Every round, developed buildings lose 2 condition points. Condition feeds rentConditionPercent():  $\geq 90 \rightarrow 100\%$ ,  $\geq 75 \rightarrow 90\%$ ,  $\geq 50 \rightarrow 75\%$ ,  $\geq 25 \rightarrow 50\%$ , below  $\rightarrow 0\%$  (closed).

When a building is not at 100%, its owner (if the strategy agrees and can afford it) pays a small maintenance fee to restore it to 100% and reset the neglect counter.

If a building goes 20+ rounds without maintenance it becomes structurally damaged: value -15%, rent -25%, condition 0, and a full renovation (at 150% cost) is required later.

**Q: Explain how the 'round' summary and winner work.**

A: displayRoundSummary() prints each player's cash, net worth, properties, railways, utilities, houses, hotels and loans each round. determineWinner() picks the surviving player with the highest net worth; if everyone is bankrupt at once it compares everyone anyway.

## 4.5 Improvement / reflection questions

**Q: What are the limitations of this project?**

A: Console only (no GUI/graphics).

Fixed 4 players and a fixed 40-square board - not configurable.

Only one active loan per player.

No save/load of a game in progress.

rand() is pseudo-random and the event decks are fixed sizes.

All sizes are compile-time constants, so large changes need recompilation.

*MEMORISE / FLAG: Say 3-4 of these confidently - examiners like self-awareness.*

**Q: How could you improve or extend the project?**

A: Add a graphical board (SDL/raylib) or colour output.

Support a variable number of players or human-vs-AI play.

Save/load game state to a file (just write the GameState struct).

Make the four strategies data-driven (function pointers) so new strategies are easy to add.

Add unit tests for calculateRent, net worth, auctions.

Use a better RNG or add statistical reporting at the end.

*MEMORISE / FLAG: Safe extra: 'The single-GameState design makes save/load trivial - you just serialize the struct.'*

**Q: What was the hardest part you had to solve?**

A: Safe answers: keeping all rule effects as temporary modifiers so base values never corrupt; making the turn-order tie-break recursive; guaranteeing loops terminate (auction caps, forced-sale guards); and keeping the round 'everyone passed GO' definition consistent.

**Q: How would you test that this code works?**

A: Seed-testing: run several games and check the final standings are printable, run with seeded srand to reproduce a game, add debug prints, and inspect invariants like: total owned squares always equal the owned counts, a skeleton test of rent/net worth, and a sanity check that games end.

## 4.6 Tricky spot questions about the code

**Q: Why does setSpecialSquare() force property.owner = -1?**

A: Because {0} zero-initialises the GameState, owner would default to 0, which is Player 1's index. Without forcing -1, GO or Tax would look like they belonged to player 0. This is exactly why the comment in board.c mentions it.

*MEMORISE / FLAG: Classic question - remember '0 is a real player index, -1 means unowned'.*

**Q: Why does the auction exclude the forced seller?**

A: Under the Anti-Speculation Act the seller must actually give the property away. If the seller could bid, it could win the auction back by paying itself (net zero), defeating the forced sale.

**Q: Why can loan-locked properties not be sold?**

A: Rule-LK 3: they are pledged as collateral for the loan. Selling them would remove the bank's security. The code checks loanLocked in sellLowValueProperty(), sellDecliningProperties(), mortgage search and the forced-sale loop (that check also stops an infinite loop).

**Q: Why does payMoney() have a Risk-Taker rescue loop, and how does it stop?**

A: When the Risk Taker's cash goes negative, before bankruptcy it sells its cheapest undeveloped property to raise money. It stops when a sale doesn't reduce the count of undeveloped properties - because everything left is loan-locked/developed and unsellable - otherwise it would loop forever.

**Q: Why is there a 'safetyLimit' in constructBuildings()?**

A: developGroup() returns 1 if it built something, so constructBuildings() keeps calling it to build the maximum in one turn. The safety limit (20 per group) guarantees the inner do-while terminates even with unusual modifiers.

**Q: What does currentCardIndex do and why not shuffle a real deck?**

A: It points into the 20-card National Event deck and wraps every draw ( $\text{index} = (\text{index} + 1) \% 20$ ). That behaves like draw-the-top-put-at-the-bottom without needing a queue/array of cards.

## 5. Cheat Sheet: Numbers Worth Remembering

---

### 5.1 Game setup

MAX\_PLAYERS 4      BOARD\_SIZE 40      MAX\_ROUNDS 500  
START\_MONEY 30,000    GO\_MONEY 2,000      JAIL\_BAIL 300  
Income Tax 15% of net worth    Community Fund 10% of property assets

### 5.2 Rent tables

Houses on a property:    0->x1    1->x2    2->x3    3->x5    4->x7    hotel->x10  
Railways owned:    1->250    2->500    3->1000    4->2000  
Utilities:    one = 4 x dice    both = 10 x dice

### 5.3 Loans and mortgages

Loan max = 75% of collateral      base interest 8% (range 1..25, step 2)  
Loan duration 20 rounds    extension +10 rounds    refinance = current rate  
Mortgage payout ~1/2 price      redeem cost = 110% of mortgage value

### 5.4 Strategy thresholds (cash in LKR)

Aggressive: reserve 1000    |    Conservative: comfort 3000, bail spare 1500  
Risk Taker: floor 500      |    Trader: floor 1200, rich line 5000, insure if >=6000

### 5.5 Economy / market

Review every 10 rounds: boom +20% value +25% rent; decline -15% value -20% rent  
Cooldown 30 rounds    inflation choices: -3, 0, 2, 5, 8, 12%  
Property ages: past 50 depreciates 5% every 5 rounds, max 30%  
Renovation = 10% of value, rent +5%    neglect 20 rounds = structural damage  
Structural damage: value -15%, rent -25%, repair at 150%

### 5.6 Insurance / disasters

Repair cost = 30% of value      Basic covers fire/flood, pays 80%  
Comprehensive 100% all risks      Business Interruption 100% + 5 lost-rent rounds  
Premiums 5% / 10% / 15%    policy 20 rounds    renew at <=10 left    warn at 3  
Disasters every 10 rounds    base weight 20 (+30 flood if monsoon, +30 riot if unrest)

### 5.7 Events cadence

Every 10 rounds: disaster, inflation, market review  
Every 15 rounds: economic event (8 types) + regional card (12 types)  
Every 20 rounds: government regulation (8 types)  
20 national event cards cycle with currentCardIndex % 20

### 5.8 Anti-Speculation Act

Max 3 undeveloped properties      violator force-auctioned after 5 consecutive rounds

## 6. Glossary of Terms to Say Confidently

---

|                      |                                                                                         |
|----------------------|-----------------------------------------------------------------------------------------|
| Struct               | A user-defined type bundling several fields (e.g. Property, Player, GameState).         |
| Enum                 | Named integer constants (PropertyGroup, SquareType, ...) that make code readable.       |
| Array decay          | Passing an array gives its first element's address; game[] talks to the real GameState. |
| Header guard         | #ifndef NAME / #define NAME / #endif stops a header being compiled twice.               |
| Prototype            | A function's signature declared in functions.h before it is defined and used.           |
| Pass by reference    | The function receives the actual object - this project does it via game[].              |
| Modifier             | A timed entry in Economy.modifiers[] that temporarily scales values/costs.              |
| Multiplier model     | All matching modifiers are multiplied together (Rule-LK 34).                            |
| Depreciation         | Properties older than 50 lose up to 30% of value in 5% steps (age damage).              |
| Condition            | Building health 0-100; drives rent and structural damage.                               |
| Structural damage    | After 20 neglected rounds a building loses value/rent until fully renovated.            |
| Mortgage             | Borrowing against one property; redeeming costs 110%.                                   |
| Collateral           | Property pledged for a loan; moves the loanLocked flag; cannot be sold.                 |
| Foreclosure          | Overdue loan - pledged properties are auctioned, player may go bankrupt.                |
| Monopoly group       | Owning every property of one colour group (enables building).                           |
| Net worth            | cash + property value + building value - outstanding loan.                              |
| Anti-Speculation Act | Limits undeveloped holdings to 3; excess is force-auctioned.                            |
| srand(time(NULL))    | Seeds rand() with the clock so each game is different.                                  |
| Modulo (%, %d)       | % returns a remainder (rand()%6 -> 0..5); %d prints an int.                             |
| Recursion            | A function calling itself - resolveGroup() for tied dice rolls.                         |

## 7. During the Viva - Exam Technique Tips

---

- ANSWER STRUCTURE: say what it does in one sentence, then how the code does it, then one number or function name. That turns a one-word answer into a strong one.
- If you don't remember exactly, say what you DO know and relate it to the project (the examiner can see your code anyway - they mostly test understanding).
- Volunteer file/function names: 'That is handled in economy.c by currentMarketValue()', 'that is Rule-LK 34, cumulative modifiers in Economy.modifiers'.
- If asked to open a function and explain, use the file-by-file section of this guide as your map.
- Never claim the code does something it does not. If unsure, offer: 'I would verify that in the code'.
- For 'improvements' questions always mention save/load (GameState is one struct), a GUI, and unit tests.
- Pause before answering, and say numbers slowly - the constants are easy marks.